DO

Document Organizer

COPY-ONLY DOCUMENT RE-ORGANIZER

TECHNICAL DESIGN

Technical Design

Architecture, data model, processing logic, storage integration and security

Document ID DO-TD-05 · **Version** 2.1 · **Date** 22-Oct-2026

Prepared by Yasin Jariwala

Audience Developers, reviewers, cloud and security teams

Document control

Document	Technical Design
Document ID	DO-TD-05
Version	2.1
Date	22-Oct-2026
Prepared by	Yasin Jariwala
Product	Document Organizer v2.1
Audience	Developers, reviewers, cloud and security teams
Classification	Public
Source	github.com/yasin679/Document_Organizer · docs/technical-design.md

Version history

Version	Date	Author	Changes
1.0	06-Oct-2026	Yasin Jariwala	First release
2.0	13-Oct-2026	Yasin Jariwala	General-purpose release: Amazon S3 and S3-compatible storage, copy between providers, container hosting; tenant ID field removed
2.1	22-Oct-2026	Yasin Jariwala	Several mapping files per job; File name cells listing several files split into one entry per file; NULL placeholders treated as blank; repeats of missing records reported as MISSING

Related documents

ID	Document
DO-UG-01	User Guide
DO-IG-02	Installation and Administration Guide
DO-QS-03	Quick Start
DO-FAQ-04	FAQ and Troubleshooting
DO-UAT-06	UAT Test Cases
DO-RN-07	Release Notes v2.1

Contents

1 Introduction 3
1.1 Purpose 3
1.2 Scope 3
1.3 Design principles 3
2 Architecture 4
2.1 Overview 4
2.2 Components 4
2.3 Technology choices 5
2.4 The storage interface 5
3 Data model 7
3.1 jobs 7
3.2 items 7
3.3 users, settings, audit 8
4 Processing logic 9
4.1 Job life cycle 9
4.2 Reading the mapping files 9
4.3 Several file names in one cell 10
4.4 Matching 10
4.5 Validation and statuses 10
4.6 Destination path 11
4.7 Copying 11
4.6a Maker-checker 12
4.7a Speed and time left 12
4.8 Resume and retry 12
5 Azure Blob Storage integration 13
5.1 Authentication 13
5.2 Least-privilege access 13
6 Amazon S3 integration 14
6.1 Requests and signing 14
6.2 Credentials 14
6.3 Least-privilege access 14
7 Security 15
8 API reference 16
9 Performance and test results 18
9.1 Automated end-to-end test 18
9.2 Additional S3 verification 18
9.3 Replay of a real job (v2.1)
9.4 Scale 19
9.5 Known limitations 19
10 Deployment 20

1 Introduction

1.1 Purpose

This document describes the design of Document Organizer v2.0: its architecture, components, data model, matching and validation logic, Azure Blob Storage and Amazon S3 integration, security controls and test results. It is the technical reference for developers, reviewers, and cloud and security teams.

1.2 Scope

- **Sources and destinations:** Azure Blob Storage, Amazon S3 and S3-compatible services, and local or network folders, in any combination.
- **Input:** one or more mapping files (`.xlsx` or `.csv`) with one row per document, accepted as exported. A File name cell may list several files.
- **Output:** a copy organized as `EmployeeCode / DocumentType / Year / FileName` (template configurable), with a validation report before copying and a reconciliation report after.
- **Out of scope for v2.0:** OCR or AI classification, single sign-on, automatic rehydration or restore of archived objects, and running several application instances against one database.

1.3 Design principles

Principle	How it is applied
Copy only	Source stores are only read. No delete or move operation on documents exists in the code. The only delete call removes the application's own write-test file during **Check access**.
No guessing	A file is copied only when exactly one valid mapping record matches it. Everything else becomes an exception with a reason.
Complete accounting	Every source file and every mapping record becomes exactly one item with a final status.
Human approval	Nothing is written to the destination until an Operator approves the scan results.
Restartable	Progress is stored per item. A restart never copies a file twice.
Provider-neutral	All storage goes through one interface, so matching, rules, reports and the UI are identical for every storage type.
Minimal footprint	Single process, no third-party packages, no database server, no cloud SDKs.

2 Architecture

2.1 Overview

Document Organizer is a single ASP.NET Core 8 process that serves a browser UI and a JSON API, runs jobs on background tasks and stores state in a local SQLite database.

```
Browser
  |  HTTPS (or http://localhost:5080) · cookie session · X-DocOrg header
  v
ASP.NET Core 8 (Kestrel) ---------------------------------------------+
  |-- wwwroot/          static UI (HTML, CSS, JavaScript; no external) |
  |-- Api/Endpoints     REST API, role checks, audit                   |
  |-- Engine/           Mapping, Scanner, Processor, JobService,       |
  |                     Reports, DemoData                              |
  |-- Storage/          IDocumentStore                                 |
  |                       +- LocalStore      (file system)             |
  |                       +- AzureBlobStore  (Blob REST API)           |
  |                       +- S3Store         (S3 REST API, SigV4)      |
  |-- Infrastructure/   SQLite (P/Invoke), xlsx/csv, users             |
  v                                                                    v
SQLite file (data/document-organizer.db)      Azure Blob · Amazon S3 · folders
```

2.2 Components

Component	File(s)	Responsibility
Host	`Program.cs`	Configuration, cookie authentication, authorization policies, security headers, CSRF guard, forwarded headers, `/healthz`, static files
API	`Api/Endpoints.cs`	All REST endpoints, input validation, role checks, audit events
Mapping	`Engine/Mapping.cs`	Reads the mapping file, detects the header row, auto-maps columns, parses dates and years, applies the folder template
Scanner	`Engine/Scanner.cs`	Lists the source, matches files to records, validates, plans destination paths, assigns statuses
Processor	`Engine/Processor.cs`	Copies READY items in batches, applies the duplicate rule, verifies integrity, writes metadata
JobService	`Engine/JobService.cs`	Runs scans and copies on background tasks; pause, resume, retry; marks interrupted jobs Paused at start-up; retention
Reports	`Engine/Reports.cs`	Validation and reconciliation reports as `.xlsx` or `.csv`

Component	File(s)	Responsibility
DemoData	`Engine/DemoData.cs`	Fictional sample files and mapping file; optionally uploaded to connections marked `UseForDemo`
Storage factory	`Storage/StorageFactory.cs`, `StorageConnectionOptions.cs`	Named, provider-typed connections; creates the store for a job location
Local store	`Storage/LocalStore.cs`	Folder access confined to a root; atomic writes through temporary files
Azure store	`Storage/AzureBlobStore.cs`	Blob REST API: Shared Key, SAS, managed identity and Azure CLI tokens
S3 store	`Storage/S3Store.cs`, `Storage/AwsCredentials.cs`	S3 REST API with Signature Version 4, multipart upload, CopyObject, AWS credential chain
SQLite	`Infrastructure/Sqlite.cs`	Thin P/Invoke wrapper over the operating-system SQLite library
Spreadsheet	`Infrastructure/Spreadsheet.cs`	Streaming `.xlsx` reader and writer, CSV reader and writer
Users	`Infrastructure/Users.cs`	Password hashing, policy, lockout, admin bootstrap

2.3 Technology choices

Choice	Reason
.NET 8 minimal API	Long-term support release; runs on Windows, Linux, macOS and in containers
No NuGet packages	Nothing to license-review or patch; builds offline
SQLite through the OS library	Built into Windows and macOS, a standard package on Linux; one file; WAL mode for concurrent reads
Azure Blob and S3 REST APIs directly	Avoids SDK dependencies; full control of signing, retries and conditional requests
Static UI without a build step	No toolchain, strict Content Security Policy, no external scripts
Container image on `aspnet:8.0`	Small runtime image, non-root user, data on a volume

2.4 The storage interface

Every source and destination implements `IDocumentStore`:

Member	Purpose
`ListAsync`	Enumerates files under the root or prefix with size, ETag, Content-MD5 (where known), access tier and last-modified time
`GetPropsAsync`	Size, ETag, Content-MD5 and user metadata of one file, or nothing if it does not exist
`OpenReadAsync`	Streams a file's content
`WriteAsync`	Writes content, refusing to replace an existing file unless overwrite is allowed

Member	Purpose
`TryServerCopyAsync`	Copies inside the storage service when source and destination allow it; returns false so the caller streams instead
`TestAsync`	Checks list and read access, or write access with a probe file

Keys are always relative to the location's root or prefix and use `/` as the separator, so the engine never sees a provider-specific path.

3 Data model

All state is in one SQLite database. Document content is never stored.

3.1 jobs

Column	Type	Meaning
`id`	TEXT PK	`JOB-YYYYMMDD-NNNN`
`name`	TEXT	Job label
`source_json`, `dest_json`	TEXT	Location type (`Local`, `AzureBlob`, `S3`), connection name, container or bucket, prefix, or path. No secrets.
`config_json`	TEXT	Column mapping per mapping file, year rule, template, matching (including splitting of several file names), duplicate rule, integrity, batch size, metadata option
`mappings_json`	TEXT	All mapping files of the job: ID, stored path, original name, rows, SHA-256, header row, headers, preview, uploader (v2.1)
`mapping_file`, `mapping_original_name`, `mapping_rows`, `mapping_sha256`, `mapping_info_json`	TEXT / INT	Summary kept for compatibility: first file's path and header, all names, total rows. Jobs from before v2.1 have only these.
`status`, `message`	TEXT	Job status and latest message
`created_by`, `created_at`, `approved_by`, `approved_at`	TEXT	Ownership and approval
`scan_started_at`, `scan_ended_at`, `started_at`, `ended_at`	TEXT	Phase timestamps (UTC)
`progress_done`, `progress_total`, `counts_json`, `discovered`	INT / TEXT	Progress and per-status counts

3.2 items

One row per source file and per unmatched mapping record.

Column	Meaning
`id`, `job_id`	Item key; foreign key to jobs (cascade delete)
`source_key`, `file_name`, `source_size`, `source_etag`, `source_md5`, `access_tier`	Source file facts captured at scan
`record_id`, `employee_code`, `employee_name`, `doc_type`, `doc_name`, `description`, `uploaded_on`, `module_code`, `submodule_code`, `year`	Values from the matched mapping record. A record split into several files gets IDs `ROW-68/1`, `ROW-68/2`.

Column	Meaning
`mapping_file`	Name of the mapping file the record came from (v2.1)
`dest_key`	Planned destination path
`status`, `reason`, `candidates_json`	Item status, plain-language reason, candidate records for AMBIGUOUS
`source_hash`, `dest_hash`, `integrity_method`	Integrity evidence after copy
`processed_at`, `processed_by`, `attempts`, `started_at`	Copy tracking used for resume and retry
`resolved_by`, `resolution_note`	Manual resolution of AMBIGUOUS items
`excluded_from`, `excluded_by`, `excluded_at`, `exclusion_note`	For EXCLUDED items: the status before exclusion, who, when (UTC) and why

Indexes: `(job_id, status)` and `(job_id, dest_key)`.

3.3 users, settings, audit

Table	Columns
`users`	`user_name` (PK, case-insensitive), `display_name`, `role`, `password_hash`, `active`, `created_at`, `last_login`
`settings`	`key`, `value` (in-app settings: storage types enabled, overwrite, parallel copies, retention)
`audit`	`id`, `ts`, `user_name`, `role`, `event`, `job_id`, `details`, `ip` (index on `ts`)
`change_requests`	`id`, `job_id`, `kind` (`exclude`/`restore`), `item_ids_json` (the items at request time), `item_count`, `summary`, `note`, `requested_by`, `requested_at`, `status` (Pending, Approved, Rejected, Withdrawn), `decided_by`, `decided_at`, `decision_note`
`exclusions`	`job_id`, `item_key` (PK together), `from_status`, `note`, `by_user`, `at`. One row per excluded item; `item_key` is `f:<source key>|<mapping file>|<record ID>` for a file (one file can have an item per record under the keep-all rule) or `r:<mapping file>|<record ID>` for a record without a file.

4 Processing logic

4.1 Job life cycle

Status	Entered when	Next
Draft	Job created; mapping file uploaded; rules changed	Scanning
Scanning	Scan started	Awaiting approval
Awaiting approval	Scan finished	Processing (approve), Draft (rules changed)
Processing	Approved, resumed or retrying	Completed, Completed with exceptions, Paused
Paused	Operator paused, or the application restarted mid-copy	Processing
Completed / Completed with exceptions	All READY items handled	Processing (retry failed)

4.2 Reading the mapping files

1. A job has one or more mapping files. Each is read as `.xlsx` (streaming XML; the first visible worksheet whose first 20 rows contain a header row, else the first visible worksheet) or `.csv` (delimiter detected; UTF-8, or Windows-1252 when the file is not valid UTF-8; a byte-order mark selects UTF-16). Uploads are checked: a file whose SHA-256 equals one already in the job (or in the same upload) is refused; a duplicate file name gets a `(n)` suffix; if any file of an upload fails, every file saved by that upload is deleted. New files always get new IDs, so a refused replacement never touches the current files.
2. The header row is the first row within the first 20 that matches at least 3 known column names.
3. Columns are auto-mapped per file by normalized name (spaces, dashes, dots and underscores removed; case ignored) using a list of common aliases. Each file keeps its own column mapping (`FileColumns` in the configuration, keyed by mapping file ID).
4. Cells that only contain a placeholder (`NULL`, `N/A`, `NA`, `#N/A`, `None`, `Nil`, `-`, `--`, `(null)`) are treated as empty.
5. Dates accept Excel serial numbers and common text formats (for example `25-Nov-2021`, `2021-11-25`, `dd/MM/yyyy`).
6. Year rule per file: with `UploadedOn`, a file whose column map has no Uploaded on column but has a Year column uses the Year column; with `YearColumn`, the reverse. Records carry `HasUploadedOnColumn` / `HasYearColumn` (also kept in ambiguous candidates, so a resolution uses the same rule).
7. Saving the configuration without `fileColumns` (as v2.0 clients do) changes only the first file's columns; other files keep their saved mapping.
8. Every record is tagged with its mapping file name. Each file's SHA-256 fingerprint and row count are stored and written to the audit log.

4.3 Several file names in one cell

The source is listed **before** matching. Then, if **Several file names in one cell** is on (the default), every record whose File name contains a comma, semicolon or line break is checked:

1. If the whole value is the name of a file in the source (optionally with a folder path), it is kept as one file name (real names can contain commas).
2. Otherwise it is split on those separators. Adjacent parts (up to five) are joined back, with their original separators, when the joined text is a source file name, so `report, final.pdf,x.pdf` gives `report, final.pdf` and `x.pdf`. Empty parts and placeholders are dropped, and repeated names within the cell are removed. A cell with only placeholders is treated as blank (INVALID).
3. Each part becomes its own record with the same employee, type, dates and descriptions, and the ID `<record ID>/<n>` (for example `ROW-68/1`, `ROW-68/2`).

Records are given an internal sequence number, so matching never depends on record IDs being unique (IDs can repeat across mapping files).

4.4 Matching

For each source file, candidate records are found in this order, stopping at the first step that finds any:

1. Exact file name.
2. Case-insensitive file name (if enabled).
3. Normalized file name: Unicode NFC, lower-case, spaces, dashes and underscores treated alike (if enabled).

If more than one record matches and some records carry a relative path, the path is used to narrow the match. If one record remains, the file is evaluated; if several remain, the file is AMBIGUOUS. Mapping records that no file matched become MISSING.

4.5 Validation and statuses

Repeated records (same normalized file name, employee code and document type as an earlier record, in any mapping file) are set aside before matching and decided afterwards: DUPLICATE if the earlier record found its file, otherwise MISSING with a reason naming the earlier record. This keeps a missing document from being counted once as missing and once as a duplicate.

Status	Rule
READY	One record; employee code, type and file name present; valid year; file size above 0; not in an archive tier; planned path unique
UNMAPPED	No record matches the file
MISSING	Record with no matching file, or a repeat of such a record
AMBIGUOUS	More than one record matches (rule `AmbiguousRule` = `Flag`, the default). With `KeepAll`, the file instead gets one item per matching record, each evaluated as if it were the only match: READY with its own destination, DUPLICATE if the record is already matched or the path is taken, or INVALID. Files are still counted once (`discovered` counts distinct source keys).
DUPLICATE	Same file, employee and type repeated in the mapping files when the first record found its file, the same record matched twice, or two items would write the same destination path

Status	Rule
INVALID	Blank required value (including placeholders such as `NULL`), no year, 0-byte file, or object in an archive tier (Azure `Archive`; S3 `GLACIER` or `DEEP_ARCHIVE`)
PROCESSED	Copied and verified
FAILED	Copy failed or source changed since scan
EXCLUDED	An exception set aside by an operator with a note; never copied; not an exception

Exclusion. UNMAPPED, MISSING, AMBIGUOUS, DUPLICATE, INVALID and FAILED items can be excluded while the job is awaiting approval, paused or completed (not while a scan or copy runs). The item keeps its data and reason; `excluded_from` keeps its status. Each exclusion is also stored in `exclusions`, and after every scan an item with the same key and the same status is excluded again before the items are saved. Restoring sets the status back and deletes the stored exclusion. A completed job is *Completed with exceptions* while any item other than PROCESSED or EXCLUDED is left; excluding or restoring recalculates it. Reasons are grouped for display and bulk actions by an SQL `CASE` over the reason text (for example `Blank FileName%` → *Blank file name*).

4.6 Destination path

The folder template uses the tokens `{EmployeeCode}`, `{EmployeeName}`, `{DocumentType}`, `{DocumentName}`, `{Year}`, `{ModuleCode}`, `{SubModuleCode}`, `{MappingFile}` (mapping file name without extension) and `{FileName}` (`<Token>` style is also accepted). Values are sanitised so they are valid as Windows folder names and as blob and object keys. The template must include `{FileName}`; an unknown token is rejected.

Example: record E00879, type Medical Certificate, uploaded 25-Nov-2021, file `e00879_final_certificate.pdf` gives `E00879/Medical Certificate/2021/e00879_final_certificate.pdf`.

4.7 Copying

1. READY items are processed in batches (default 5,000) with up to 8 parallel copies (configurable 1 to 64).
2. Before each batch the items' `started_at` is recorded.
3. The duplicate rule is applied if the destination exists: Skip (default), Rename (`_dup1` to `_dup999`), Compare hash (skip if SHA-256 equal, else rename) or Overwrite (only if allowed by an administrator).
4. **Server-side copy** is tried first: - Azure to Azure in the same storage account: Copy Blob (`x-ms-copy-source`) with `x-ms-source-if-match` set to the source ETag. - S3 to S3 on the same connection: CopyObject (`x-amz-copy-source`) with `x-amz-copy-source-if-match`. Objects over 5 GB fall back to streaming. - Folder to folder: file copy to a temporary name, then rename.
5. Otherwise the content is **streamed** from source to destination after re-checking the source ETag, so a file that changed after the scan fails instead of copying the wrong version. - Azure destination: single Put Blob up to 8 MiB, otherwise Put Block / Put Block List. - S3 destination: single PutObject up to 8 MiB with Content-MD5, otherwise multipart upload in 8 MiB parts (aborted on failure). - Folder destination: written to a `.partial-` temporary file and renamed when complete.
6. Creating a file that must not overwrite uses a conditional request (`If-None-Match: *`) on Azure and S3, so two writers cannot both succeed.
7. Integrity is verified by SHA-256 (read both sides) or Content-MD5 (Azure's stored MD5; for S3, the ETag of single-part objects without KMS or customer-key encryption, which equals the MD5).

8. Cloud destinations get metadata `docorgjobid`, `docorgrecordid` and `docorgemployeecode`. The source object's own metadata is kept if selected; names are adjusted to each provider's rules (Azure metadata names must be valid identifiers).
9. The item is marked PROCESSED or FAILED with the reason.

4.6a Maker-checker

Setting `RequireSecondApprover` (default off). `jobs.makers_json` lists everyone who created or changed the job (create, mapping files, rules, scan, resolve, exclude, restore, submit). `submitted_by` / `submitted_at` / `submit_note` mark a job waiting for a checker; while set and the job is *Awaiting approval*, every changing endpoint refuses. Approve and reject require a user not in `makers_json` (case-insensitive) with role Operator or Administrator. After approval, exclude and restore create a `change_requests` row holding the item IDs selected at request time; approving it (by anyone but the requester) applies the change to those items that are still eligible. Retry and resume are not gated: they only carry out what was already approved.

4.7a Speed and time left

- **Hash while copying.** Folder to folder with an integrity check, the source is streamed to the destination through a hashing stream (1 MB reads), so each source file is read once; the destination is then read once to verify. Before, the source was read twice (copy, then hash). Cloud server-side copies are unchanged; other streaming copies (for example folder to S3) also hash the source on the way.
- **Results saved every 2 seconds** or every 250 files, whichever comes first (was every 250 files only).
- **Time left.** A sliding-window meter (last two minutes) of items and bytes. Copying: time left = bytes still to copy ÷ bytes per second (falls back to files ÷ files per second when sizes are unknown). Scanning: per step, items left ÷ items per second, for steps with a known total. Live figures are kept in memory while a job runs and returned by `GET /jobs/{id}` as `progress` (`done`, `total`, `bytesDone`, `bytesTotal`, `copied`, `skipped`, `failed`, `rate`, `bytesRate`, `etaSeconds`, `elapsedSeconds`, `live`) and `scan` (`phase`, `done`, `total`, `rate`, `etaSeconds`, `phaseElapsedSeconds`, `elapsedSeconds`). `copyResult` gives the copy step's processed, skipped and failed counts.

4.8 Resume and retry

At start-up, any job left in Processing is marked Paused. On resume, items with `started_at` set but not finished are checked: if the destination exists and its SHA-256 equals the source, the item is marked PROCESSED without copying again. **Retry failed** refreshes each failed item's ETag and size from the source and copies only those items.

5 Azure Blob Storage integration

5.1 Authentication

Mode	Mechanism
Shared Key	Connection string with `AccountKey`; HMAC-SHA256 request signing
SAS	SAS token appended to each request
Managed identity	Token from `IDENTITY_ENDPOINT` (App Service, Container Apps) or IMDS (VM), resource `https://storage.azure.com/`
Azure CLI	Token from `az account get-access-token`, refreshed before expiry

REST API version `2021-12-02`. Listing uses pages of 5,000 blobs. Throttling and transient errors (429, 500, 502, 503, 504, timeouts) are retried up to 7 times with exponential back-off, honouring `Retry-After`.

5.2 Least-privilege access

Container	Role	Operations used
Source	Storage Blob Data Reader	List Blobs, Get Blob Properties, Get Blob
Destination	Storage Blob Data Contributor	Create Container, List Blobs, Put Blob, Put Block, Put Block List, Copy Blob, Get Blob Properties, Get Blob, Delete Blob (write-test file only)

6 Amazon S3 integration

6.1 Requests and signing

- Every request is signed with **AWS Signature Version 4** (header-based, `s3` service), including the payload SHA-256. Keys are encoded per RFC 3986 and sent exactly as signed.
- **Addressing:** virtual-hosted (`bucket.s3.region.amazonaws.com`) for AWS; path-style (`endpoint/bucket/key`) when `ServiceUrl` is set or `ForcePathStyle` is true. Buckets with dots use path-style over HTTPS to match the TLS certificate.
- **Listing:** ListObjectsV2, 1,000 keys per page with continuation tokens. Zero-byte keys ending in `/` (folder placeholders) are skipped.
- **Errors:** XML error codes are surfaced as-is (for example `NoSuchBucket`, `AccessDenied`). A wrong region is reported with the bucket's actual region from `x-amz-bucket-region`. CopyObject and CompleteMultipartUpload responses are checked for an error document returned with HTTP 200.
- **Retries:** as for Azure (429, 500, 502, 503 `SlowDown`, 504, network errors).
- **Buckets:** the write check creates a missing destination bucket (with a `LocationConstraint` outside `us-east-1`) when the credentials allow it.

6.2 Credentials

`Auth`	Source
`AccessKey`	`AccessKeyId`, `SecretAccessKey`, optional `SessionToken` from configuration
`Environment`	`AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `AWS_SESSION_TOKEN`
`Profile`	Static keys in `~/.aws/credentials` (or `AWS_SHARED_CREDENTIALS_FILE`)
`WebIdentity`	`AssumeRoleWithWebIdentity` at the regional STS endpoint with `AWS_WEB_IDENTITY_TOKEN_FILE` and `AWS_ROLE_ARN` (EKS IRSA)
`Container`	`AWS_CONTAINER_CREDENTIALS_RELATIVE_URI` or `_FULL_URI` (ECS task role, EKS Pod Identity)
`InstanceProfile`	EC2 instance metadata service, IMDSv2 token first
`Default`	The first available of the above, in that order

Temporary credentials are cached and refreshed five minutes before they expire.

6.3 Least-privilege access

Bucket	Actions used
Source	`s3:ListBucket`, `s3:GetObject`
Destination	`s3:ListBucket`, `s3:GetObject`, `s3:PutObject` (includes CopyObject and multipart), `s3:AbortMultipartUpload`, `s3:DeleteObject` (write-test file only), optional `s3:CreateBucket`

7 Security

Area	Control
Authentication	Local accounts; PBKDF2-SHA256, 210,000 iterations; 10-character, 3-class policy; sign-in blocked for 10 minutes after 5 failures per user name and IP address; server-side sessions (table `sessions`) ended by sign-out, password change or reset; every change to one job is handled one request at a time (per-job lock), and approval can carry the job `version` so a stale page cannot approve a changed job (HTTP 409)
Session	HttpOnly, SameSite=Strict cookie, Secure over HTTPS; sliding inactivity timeout (default 30 minutes)
Authorization	Policies View, Operate and Admin mapped to Viewer, Operator and Administrator on every endpoint; disabled users lose access on their next request
CSRF	State-changing API requests must carry the header `X-DocOrg: 1`
Browser hardening	Content-Security-Policy `default-src 'self'`, `X-Frame-Options: DENY`, `nosniff`, no external assets
Secrets	Cloud credentials only in configuration or environment; never in the database, API responses, reports or logs
Paths	Local paths confined to `AllowedLocalRoots`; destination inside source refused; keys cannot escape a folder root
Reports	CSV cells beginning with `=`, `+`, `-` or `@` are escaped
Data	No document content in the database; documents never sent to external services
Audit	Append-only audit table covering sign-in, jobs, mapping, rules, scan, resolution, approval, processing, retry, export, settings and users
Container	Runs as a non-root user; only port 8080 exposed

8 API reference

All endpoints are under `/api` except `/healthz`. Roles: V = Viewer, O = Operator, A = Administrator.

Method and path	Role	Purpose
`GET /healthz`	none	Liveness check
`POST /auth/login`, `/auth/logout`; `GET /auth/me`	any	Session
`POST /account/password`	V	Change own password
`GET /meta`, `/dashboard`	V	Connections (name, provider, sign-in method, endpoint), enabled storage types, totals
`POST /connections/test`	O	Check access to a location
`GET /jobs`, `/jobs/{id}`	V	List and detail
`POST /jobs`; `DELETE /jobs/{id}`	O	Create; delete (own Draft, or any for A)
`POST /jobs/{id}/mapping`	O	Replace all mapping files with the uploaded file(s)
`POST /jobs/{id}/mappings`	O	Add one or more mapping files (multipart, any field name)
`DELETE /jobs/{id}/mappings/{mappingId}`	O	Remove one mapping file
`POST /jobs/{id}/mapping/demo`	O	Use the demo mapping file
`PUT /jobs/{id}/config`	O	Save rules
`POST /jobs/{id}/scan`	O	Start scan
`GET /jobs/{id}/items?status=&q=&group=&page=`, `/jobs/{id}/tree`	V	Results (filter by status, search, reason group) and proposed structure
`POST /jobs/{id}/items/{itemId}/resolve`	O	Resolve ambiguous file
`GET /jobs/{id}/reasons[?excluded=true]`	V	Exceptions (or excluded items) counted by status and reason group
`POST /jobs/{id}/items/exclude`	O	Exclude: `{ "ids": [..], "note": "..." }` or `{ "allInView": true, "status", "q", "group", "note" }`; note required
`POST /jobs/{id}/items/restore`	O	Restore: `{ "ids": [..] }` or `{ "allInView": true, "q", "group" }`
`POST /jobs/{id}/approve`, `/pause`, `/resume`, `/retry-failed`	O	Processing control (with maker-checker on, approve needs a submitted job and a user who is not a maker)
`POST /jobs/{id}/submit` `{note}`, `/withdraw`, `/reject` `{note}`	O	Maker-checker: submit for approval, withdraw (submitter or A), reject with a reason (not a maker)

Method and path	Role	Purpose
`GET /jobs/{id}/requests`; `POST /jobs/{id}/requests/{rid}/approve`, `/requests/{rid}/reject` `{note}`	V / O	Change requests after approval; the requester's own reject is a withdrawal
`GET /jobs/{id}/report?kind=&format=`	V	Validation or reconciliation report
`GET /audit`	V	Audit log
`GET`, `PUT /settings`; `GET`, `POST`, `PUT /users`	A	Administration
`POST /demo`	O	Create demo data (when enabled)

A location in the API is `{ "type": "Local" | "AzureBlob" | "S3", "path": "...", "connection": "...", "container": "...", "prefix": "..." }`; for S3, `container` holds the bucket name.

9 Performance and test results

9.1 Automated end-to-end test

`tests/e2e-mapping.mjs` (v2.1) checks splitting of several file names in one cell, a real file name containing a comma, `NULL` placeholders, repeats of missing records, two mapping files with different column names in one job, removing and adding mapping files, the `{MappingFile}` token and the Mapping File report column (24 checks).

`tests/e2e.mjs` runs the full flow and checks counts per status, matching options, the reference example path, ambiguity resolution, both reports, integrity hashes, the CSRF guard and role restrictions. For v2.0 it runs seven jobs:

Job	Copy method	Result
Local folder → local folder	File copy	Pass
Azure (Azurite) → Azure	Server-side Copy Blob	Pass
S3 → S3	Server-side CopyObject	Pass
Azure → local folder	Streaming	Pass
S3 → local folder	Streaming	Pass
Azure → S3	Streaming (multipart over 8 MiB)	Pass
S3 → Azure	Streaming	Pass

159 checks pass. The same suite runs in GitHub Actions on every push.

9.2 Additional S3 verification

Test	Result
Every request against an S3 emulator that enforces Signature V4 (about 10,000 requests, including multipart, CopyObject, bucket creation in `ap-south-1` and credentials from environment variables)	All accepted; a wrong secret is rejected with `SignatureDoesNotMatch`
20 MB file copied to S3	Multipart upload in 3 parts; content byte-identical
Key with spaces, accents and `+&=@#`	Stored and read back under the exact key
Second run into the same destination	Every item DUPLICATE; nothing overwritten
Copied objects	`docorgjobid`, `docorgrecordid`, `docorgemployeecode` metadata present

9.3 Replay of a real job (v2.1)

A production job's reconciliation report (51,300 mapping records, 107,799 source files, a local folder) was rebuilt with placeholder file contents and scanned with v2.0 and v2.1. v2.0 reproduced the original report exactly.

Status	v2.0	v2.1	Why
Processed	41,296	55,948	5,840 records listed 14,657 files in one cell; they are now split and matched
Unmapped	66,502	51,850	The same 14,652 files now have a record
Missing	8,351	923	Multi-file cells no longer counted as one missing file; 1,591 `NULL` file names are now INVALID
Invalid	1,639	3,231	1,592 `NULL` file names (blank) added
Duplicate	14	15	4 repeats of missing records moved to MISSING / INVALID; 5 files listed twice after splitting

Scan time: 4.8 seconds; copy of 55,948 files: 7.8 seconds (local disk).

9.4 Scale

Measure	Result
Mapping file of 66,000 rows with 60,040 source files (local folder)	Read, matched and validated in about 3 seconds
v1.0 scale test: 176,000-row Excel file and 60,000 files	Read 3 s, scan 15 s, copy with SHA-256 30 s; restart mid-copy and resume gave exactly 60,000 processed, no duplicates

9.5 Known limitations

Limitation	Plan
Tested against emulators; real Azure (Azure CLI, managed identity) and real AWS (IAM roles) to be confirmed in UAT	UAT cases DO-UAT-06
Local accounts only	Single sign-on (OpenID Connect) on the roadmap
Archived objects not rehydrated or restored	By design; reported as INVALID
Single instance; not designed for a web farm	Run one instance per data volume
Server-side copy only within one Azure account or one S3 connection	Other combinations stream through the application

10 Deployment

Option	Use
Laptop (Windows, Linux, macOS)	`start.cmd` or `start.sh`, http://localhost:5080, Azure CLI or AWS profile sign-in
Container (Docker, Kubernetes, Azure Container Apps, AWS ECS or App Runner)	Image from the `Dockerfile`, volume at `/app/data`, HTTPS at the ingress, managed identity or IAM role
Azure App Service or IIS	Published application, HTTPS binding, managed identity, `AllowedLocalRoots` set, demo data off
Linux server	systemd service behind a reverse proxy with HTTPS

See the Installation and Administration Guide (DO-IG-02) for step-by-step instructions.